

The background of the slide is a photograph of a person sitting and reading a book. The person's hands and part of their torso are visible, wearing a light-colored, textured sweater. The book is open, and the pages are slightly yellowed. To the left of the book, there is a white cup, likely containing coffee, on a surface. The lighting is soft and warm, creating a cozy atmosphere. In the top left corner, there is a small orange rectangular graphic element.

Lecture 3

Getting Familiar with Flutter Widgets

By Mukhammadali Khayotov

Contents

Introduction to Flutter

Importance of Widgets in Flutter

Brief Overview

Overview of Flutter's Widget Tree and Component Hierarchy

Building User Interfaces with Core Flutter Widgets

Creating Responsive Layouts using Flexbox and Media Queries

LayoutBuilder

Impeller Rendering Engine

Support for WebAssembly (Wasm)



Introduction to Flutter

- Flutter is an open-source UI software development kit created by Google. It is used to develop applications for multiple platforms including iOS, Android, the web, Windows, MacOS, and Linux from a single codebase.
 - Cross-Platform development
 - Close-to-native performance
 - Fast, consistent, and customizable rendering (not platform Specific Rendering)

What is Cross Platform?

What do we mean by close to native performance?

Why React Native is not considered as close to native?

Which one actually uses Android's or iOS's native rendering solutions?

React Native or Flutter?

Based on their core architecture, **Flutter is generally considered to be closer to native performance.**

- **Flutter's Direct Rendering:** Flutter does not use the native UI components of the operating system (iOS or Android). Instead, it brings its own high-performance rendering engine called **Skia** to draw every pixel on the screen. This direct control over rendering eliminates the need for a "bridge" to translate code, resulting in consistently high frame rates and smooth animations that are often indistinguishable from a native application.
- **React Native's Bridge Architecture:** Traditionally, React Native has used a JavaScript "bridge" to communicate between the JavaScript code developers write and the native platform's UI elements. This back-and-forth communication can create a bottleneck, especially for complex animations or heavy data processing, which can sometimes lead to performance that is a step below true native.

What is a Widget?

- Building blocks of Flutter application
- Widgets are immutable
- Widget is an object model that unifies all views such as view controllers, layouts and other properties

What are Widgets in Flutter

In Flutter, **widgets** are the fundamental building blocks of the user interface.

Think of them like LEGO bricks. You combine smaller, single-purpose widgets to build more complex widgets, which eventually form your entire app screen. In Flutter, literally everything is a widget—from a button or text to the layout structure (like rows and columns) and even invisible elements like padding.



Importance of Widgets in Flutter

- Widgets are the fundamental building blocks of any application built by Flutter. For several reasons:
 - Structure
 - [Widgets provide the basic structure of a Flutter app. Each element on a screen of a Flutter app is a widget, and the structure of the code of an app is a tree of widgets.](#)
 - State Management
 - [Widgets play a critical role in managing the state of any Flutter app. Whenever a widget changes its state, it rebuilds its description.](#)
 - UI Design
 - [Widgets describe what their view should look like given their current configuration and state. They can be used to create layouts, display text and images, and interact with users.](#)
 - Versatility
 - [There are many types of widgets available in Flutter, including layout widgets, text widgets, input widgets, assets, images, and icon widgets.](#) This versatility allows developers to create a wide range of UI designs.
 - Reusability
 - Widgets can be reused across different parts of an application, promoting code reusability and consistency.

Stateless Widgets

- Stateless Widgets do not have any mutable states. They are immutable (i.e.: their appearance and property do not change while the widget is alive)
 - **Immutable** → cannot be modified nor redrawn once the app is running.
 - **User Interface** → The main purpose of Stateless Widgets is to convey information displaying through UI. (i.e.: they are independent of user's input)
- Some examples: Text, IconButton, AppBar and etc...

Stateful Widgets

- Stateful Widgets in Flutter are dynamic and can be changed during runtime.
 - **Mutable State** → there is a track which stateful widgets need to follow. It can be modified according to the user's input. Also, a track of the previous state value is taken into account as it needs to be rebuild with new changes.
(updates after changes)
 - **Check Update** → A stateful widget usually checks for 2 things: whether the state is changed or it is up to date and does not need to be updated.

Wait...

Several slides back, we mentioned that Widgets are immutable and in the previous slide we are talking about a STATEFUL WIDGET:)

How can this be explained?

Any ideas?

Answer

It is indeed immutable, Therefore we have STATEs! to mimic the mutability. Once the state changes that very widget is rebuild with necessary changes.

Stateful Widgets

A basic structure of a Stateful Widget

```
class MyApp extends StatefulWidget {  
  const MyApp ( {Key? key}) : super (key: key);  
  @override  
  State<MyApp> createState () => _MyAppState ();  
}  
class _MyAppState extends State<MyApp> {  
  @override  
  Widget build (BuildContext context) {  
    return Container ();  
  }  
}
```

The MyApp is a class that extends StatefulWidget. A generic method createState() creates a mutable state for the widget at a particular location and returns another class that is extended by the previous state. The _MyAppState class extends another state.

Special Methods: There are various methods provided by the Stateful class to work with, such as BuildContext, SetState(), initState(), and didChangeDependencies()¹.

Time to play

Study the very next slide, and try to guess the output.

We need some volunteers to draw it on the Smart Panel.

Other students can share their ideas and help our volunteers.

Understanding the Widget Tree

Introduction to Widgets

```
Card(  
  . margin: EdgeInsets.all(8.0),  
  . elevation: 4.0,  
  . child: Column(  
    . crossAxisAlignment: CrossAxisAlignment.start,  
    . children: <Widget>[  
      . Padding(  
        . padding: const EdgeInsets.all(8.0),  
        . child: Text("Average Daily Mood", style: TextStyle(fontSize: 17.0, fontWeight: FontWeight.bold),),  
      ),  
      . Padding(  
        . padding: const EdgeInsets.all(8.0),  
        . child: Row(  
          . crossAxisAlignment: CrossAxisAlignment.center,  
          . mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
          . children: <Widget>[  
            . BuildMoodVerticalBar(  
              . moodIcon: _sundayAverageMood,  
              . dayAbbreviation: "S",  
            ),  
            . Padding(padding: EdgeInsets.only(left: 16.0)),  
            . BuildMoodVerticalBar(  
              . moodIcon: _mondayAverageMood,  
              . dayAbbreviation: "M",  
            ),  
            . Padding(padding: EdgeInsets.only(left: 16.0)),  
            . BuildMoodVerticalBar(  
              . moodIcon: _tuesdayAverageMood,  
              . dayAbbreviation: "T",  
            ),  
            . Padding(padding: EdgeInsets.only(left: 16.0)),  
            . BuildMoodVerticalBar(  
              . moodIcon: _wednesdayAverageMood,  
              . dayAbbreviation: "W",  
            ),  
            . Padding(padding: EdgeInsets.only(left: 16.0)),  
            . BuildMoodVerticalBar(  
              . moodIcon: _thursdayAverageMood,  
              . dayAbbreviation: "T",  
            ),  
            . Padding(padding: EdgeInsets.only(left: 16.0)),  
            . BuildMoodVerticalBar(  
              . moodIcon: _fridayAverageMood,  
              . dayAbbreviation: "F",  
            ),  
            . Padding(padding: EdgeInsets.only(left: 16.0)),  
            . BuildMoodVerticalBar(  

```



```
Widget build(BuildContext context) {  
  . return SizedBox(  
    . height: 175.0,  
    . width: MediaQuery.of(context).size.width / 14,  
    . child: Column(  
      . mainAxisAlignment: MainAxisAlignment.end,  
      . children: <Widget>[  
        . BuildMoodIcon(moodIcon: moodIcon, height: 24.0, width: 24.0),  
        . Padding(padding: EdgeInsets.only(top: 8.0)),  
        . Container(  
          . width: MediaQuery.of(context).size.width / 14,  
          . height: moodChartPercentage(moodIcon) * 100,  
          . color: moodColor(moodIcon),  
        ),  
        . Padding(padding: EdgeInsets.only(bottom: 8.0)),  
        . Text(dayAbbreviation),  
      ],  
    ),  
  );  
}
```

Widget Tree

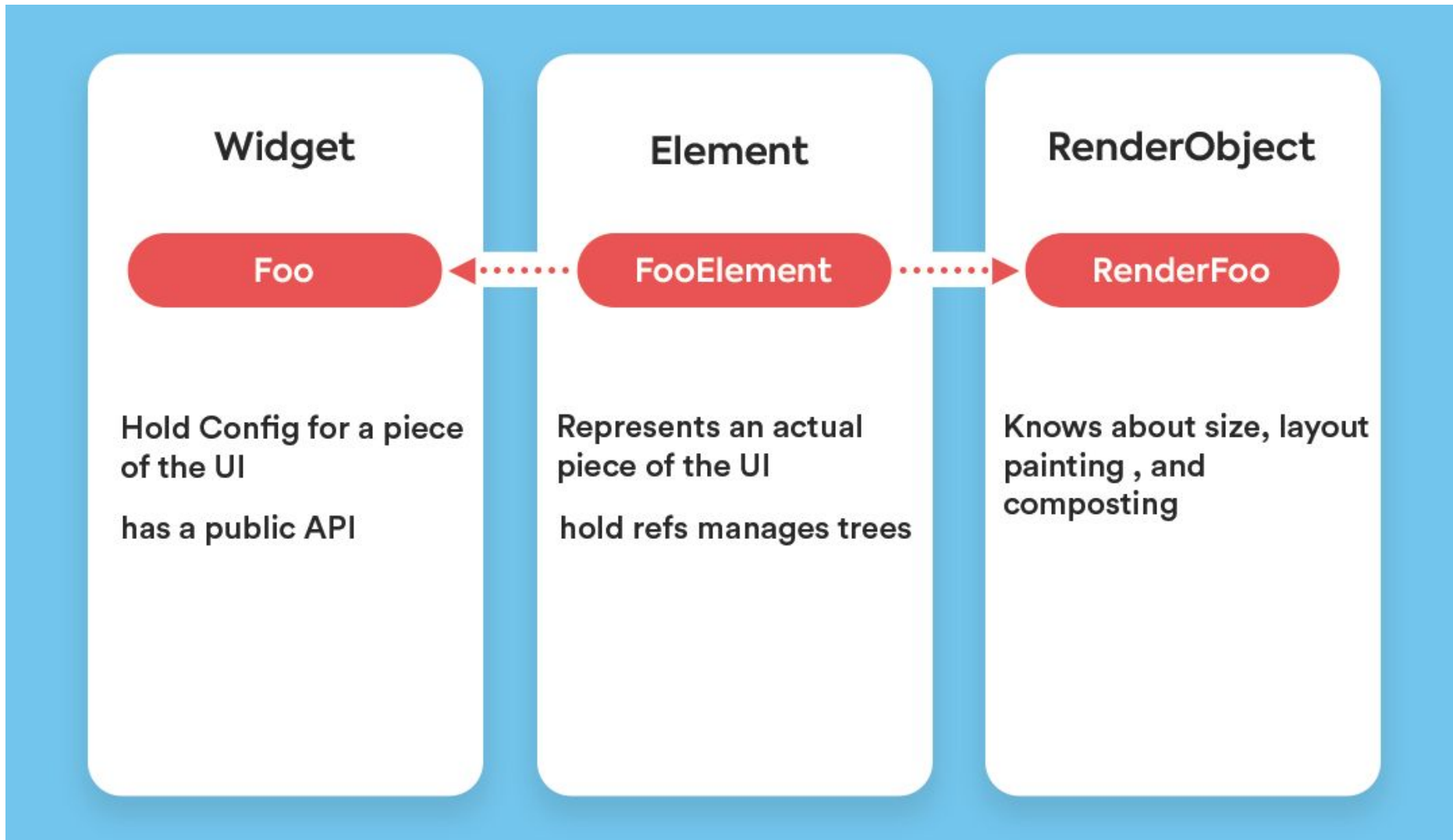
Think of it like a family tree for your user interface. At the very top, you have your main ancestor, which is usually the `MaterialApp` widget that represents your whole application.

Everything else branches off from there. That `MaterialApp` might have a child, say, a `Scaffold` (which is just a standard page layout). That `Scaffold` then has its own children: an `AppBar` for the title bar and a `Body` for the main content. The `AppBar`'s child would be a `Text` widget for the title, and the `Body`'s child could be a `Center` widget, which in turn has its own `Text` widget child.

So you end up with this hierarchy where widgets are nested inside each other. That entire structure is the "widget tree."

It's basically the blueprint Flutter follows to draw the UI. When you change something, Flutter just looks at the tree to see which part needs a fresh coat of paint, making it really efficient.

How Widgets are rendered on Screen



In short, Flutter uses **three separate "trees"** behind the scenes to make rendering fast and efficient. You, the developer, only work with the Widget tree.

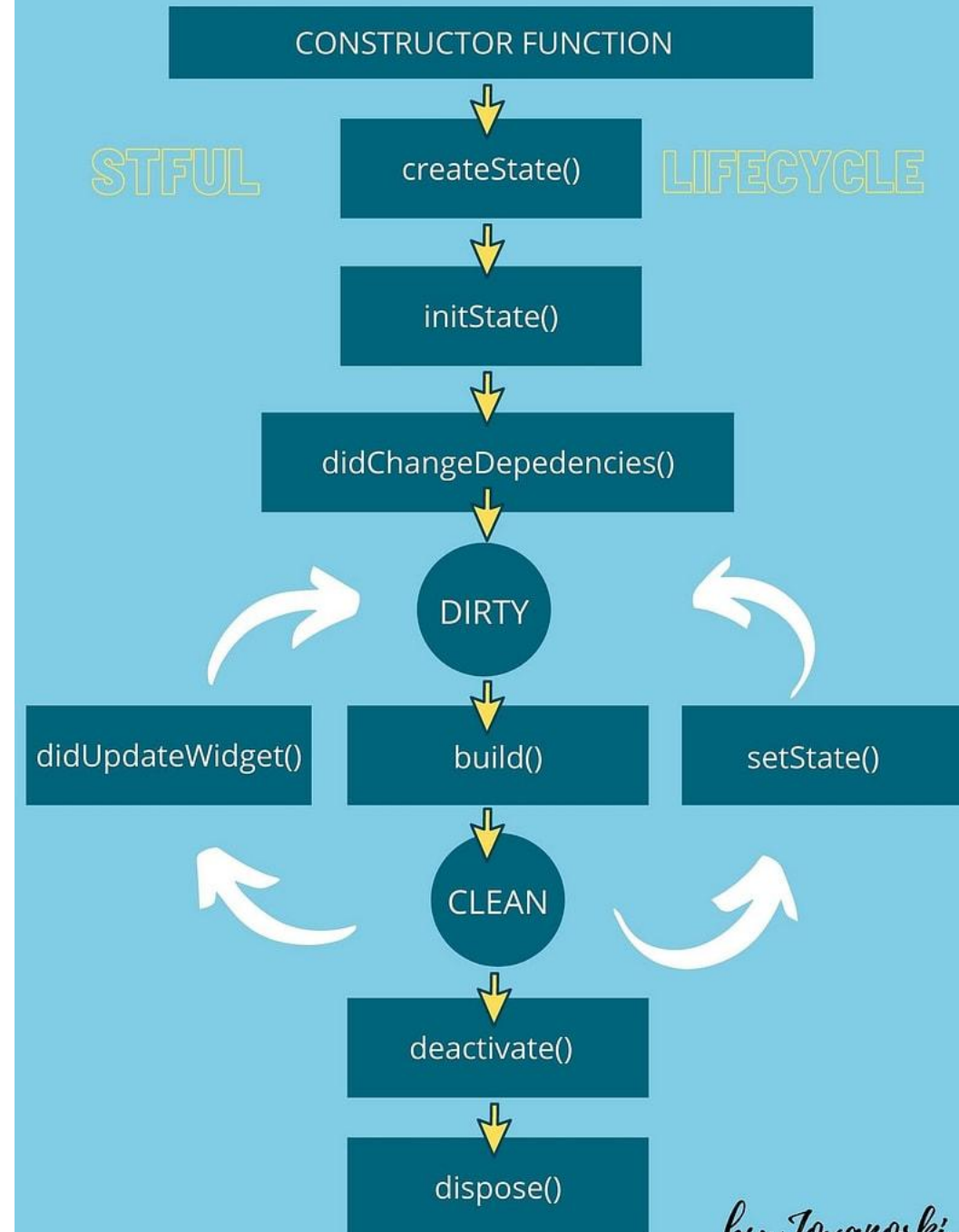
Here's what each part does, using a simple construction analogy:

1. **Widget (The Blueprint):** This is the code you write. It's an immutable, lightweight blueprint that describes how a piece of the UI should look and be configured.
2. **Element (The Project Manager):** This is the "smart" layer. The Element is the actual instance of the widget living in the tree. It manages the lifecycle and holds a reference to both the blueprint (Widget) and the worker (RenderObject). Its main job is to efficiently compare old and new blueprints to decide what needs to change on screen.
3. **RenderObject (The Construction Worker):** This is the object that does all the heavy lifting. It handles the actual size, layout, and painting of the pixels on the canvas. It knows nothing about the blueprint; it just follows orders from the Element.

So the flow is: Your **Widget** (blueprint) tells the **Element** (manager) what to create, and the **Element** directs the **RenderObject** (worker) to do the actual drawing. This separation is key to Flutter's high performance.

Widget Lifecycle

- Construction
- Initialization
- Build
- State Updates
- Rebuilding
- Deactivation
- Disposal



PROBLEMS 19 OUTPUT TERMINAL DEBUG CONSOLE

```
Restarted application in 384ms.  
flutter: from constructor  
flutter: from create state  
flutter: from initState  
flutter: from didChangeDependencies  
flutter: from constructor  
flutter: from didUpdateWidget  
Reloaded 1 of 569 libraries in 272ms.
```

Brief Explanation of the Slide

The lifecycle is the sequence of stages a widget goes through from its creation to its destruction. Think of it as the widget's "life story."

There are four main phases:

1. Creation (One-Time Setup) This happens only once when the widget is first added to the screen.

- **Constructor:** The widget's blueprint is created.
- **`createState()`:** Flutter creates a separate `State` object to hold all the mutable data for the widget.
- **`initState()`:** This is your chance to do one-time setup work, like fetching data from an API or initializing controllers. It's called only once in the widget's entire life.

2. Build (Painting the UI)

- **`build()`:** This is the most important method. It's responsible for drawing the widget on the screen by returning a tree of other widgets. This method is called frequently—initially after `initState()` and every time the widget needs to be updated.

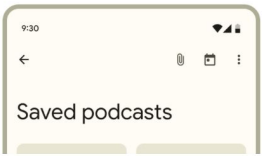
3. State Updates & Rebuilding (Reacting to Change) This is the loop in the middle of the diagram where the widget spends most of its life. The UI is updated ("rebuilt") in two main ways:

- **`setState()`:** Called from within the widget to signal that its *internal* state has changed (e.g., a counter increases). This marks the widget as "Dirty" and schedules a `build()` to redraw it with the new information.
- **`didUpdateWidget()`:** Called when the widget's configuration changes from its *parent* (e.g., the parent passes it a new title). This allows the widget to react to external data changes before a `build()` is triggered.

4. Disposal (Cleaning Up) This happens only once when the widget is permanently removed from the screen.

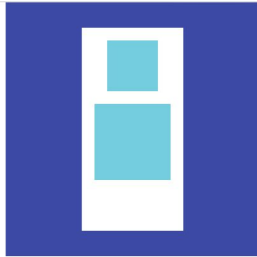
- **`deactivate()`:** The widget is removed from the tree.
- **`dispose()`:** This is your chance to do final cleanup work to prevent memory leaks, such as closing data streams or disposing of controllers. It is the opposite of `initState()`.

Common Widgets



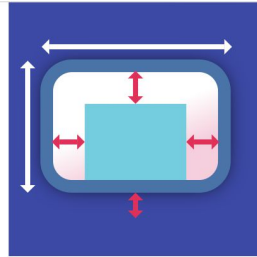
AppBar

Container that displays content and actions at the top of a screen.



Column

Layout a list of child widgets in the vertical direction.



Container

A convenience widget that combines common painting, positioning, and sizing widgets.



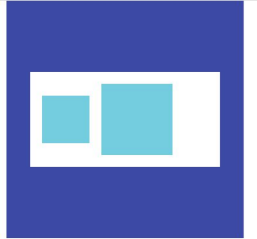
Image

A widget that displays an image.



Placeholder

A widget that draws a box that represents where other widgets will one day be added.



Row

Layout a list of child widgets in the horizontal direction.



ElevatedButton

A Material Design elevated button. A filled button whose material elevates when pressed.



FlutterLogo

The Flutter logo, in widget form. This widget respects the IconTheme.



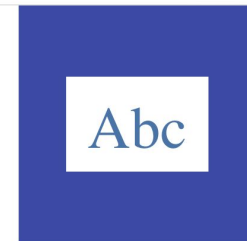
Icon

A Material Design icon.



Scaffold

Implements the basic Material Design visual layout structure. This class provides APIs for showing drawers, snack bars, and bottom sheets.



Text

A run of text with a single style.

Common Widgets *(Text, Row and Column)*

- Text is a widget which displays a styled text

```
Text(  
  'Hello, Flutter!',  
  style: TextStyle(fontSize: 32, color: Colors.blue),  
)
```

- Row and Column are flexible layout widgets that allow you align children widgets horizontally or vertically

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceAround,  
  children: <Widget>[  
    Text('Hello!'),  
    Text('Flutter!'),  
  ],  
)
```

Common Widgets *(Container and Icon)*

- Container allows common painting, positioning and sizing widgets

```
Container(  
  margin: const EdgeInsets.all(10.0),  
  color: Colors.amber[600],  
  width: 48.0,  
  height: 48.0,  
)
```

EdgeInsets $\square$ an immutable offset set for all 4 directions
min value: 50 (makes it lighter)
max value: 900 (makes it darker)
average is usually 500 (the actual color we mean)

- Icon is a an image or graphical that is displayed on a device's menu through which we can navigate to the application

```
Icon(  
  Icons.star,  
  color: Colors.red[500],  
)
```

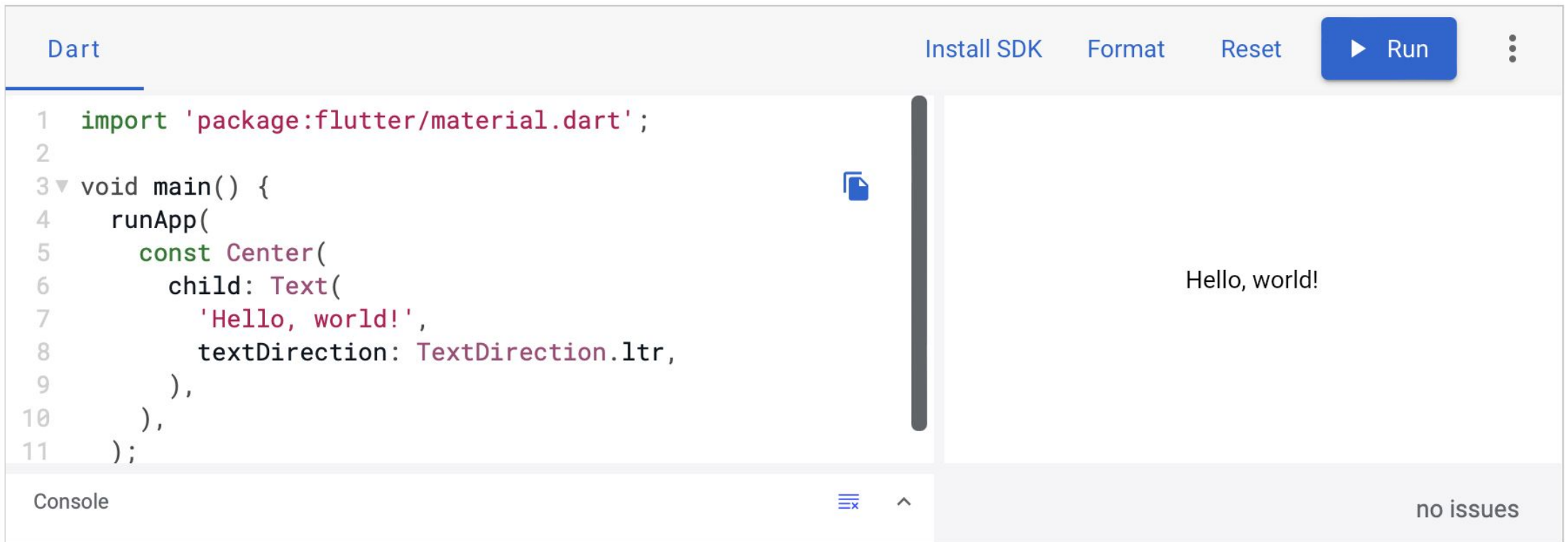
Colors.red[300] Colors.red.shade300	0xFFE57373
Colors.red[400] Colors.red.shade400	0xFFEF5350
Colors.red	0xFFFF4436
Colors.red[600] Colors.red.shade600	0xFFE53935

Common Widgets *(Descriptions)*

- Flutter comes with a suite of powerful basic widgets, of which the following are commonly used:
- [Text](#)
 - The Text widget lets you create a run of styled text within your application.
- [Row](#), [Column](#)
 - These flex widgets let you create flexible layouts in both the horizontal (Row) and vertical (Column) directions. The design of these objects is based on the web's flexbox layout model.
- [Stack](#)
 - Instead of being linearly oriented (either horizontally or vertically), a Stack widget lets you place widgets on top of each other in paint order. You can then use the [Positioned](#) widget on children of a Stack to position them relative to the top, right, bottom, or left edge of the stack. Stacks are based on the web's absolute positioning layout model.
- [Container](#)
 - The Container widget lets you create a rectangular visual element. A container can be decorated with a [BoxDecoration](#), such as a background, a border, or a shadow. A Container can also have margins, padding, and constraints applied to its size. In addition, a Container can be transformed in three-dimensional space using a matrix.

Building User Interfaces with Flutter

A "Hello World" app



Building User Interfaces with Flutter

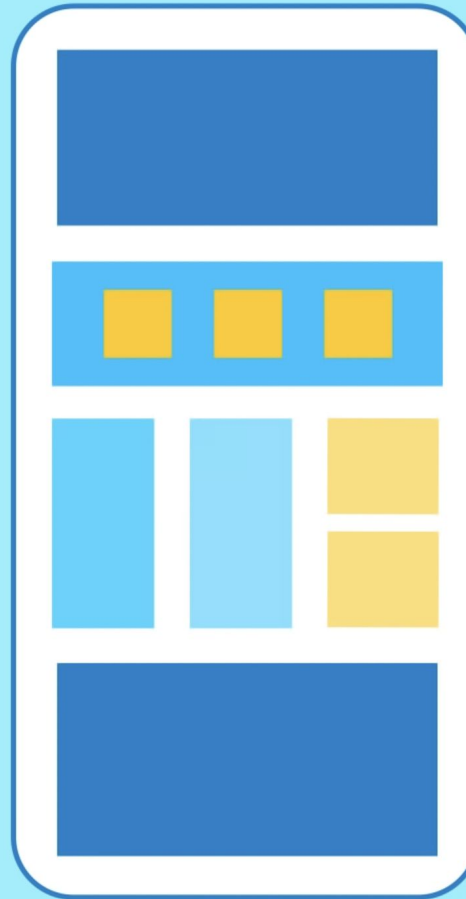
- Now let's head to Flutter Documentation to examine some examples for building UI with Flutter core Widgets: [link](#)

Creating Responsive Layouts using FlexBox and MediaQueries

- In Flutter, the term Flexbox is implemented through the Flex, Expanded, and Flexible widgets.
- **Flex:** The Flex widget allows you to control the axis along which the children are placed (horizontal or vertical). This is referred to as the main axis. If you know the main axis in advance, then consider using a Row (if it's horizontal) or Column (if it's vertical) instead, because that will be less verbose.

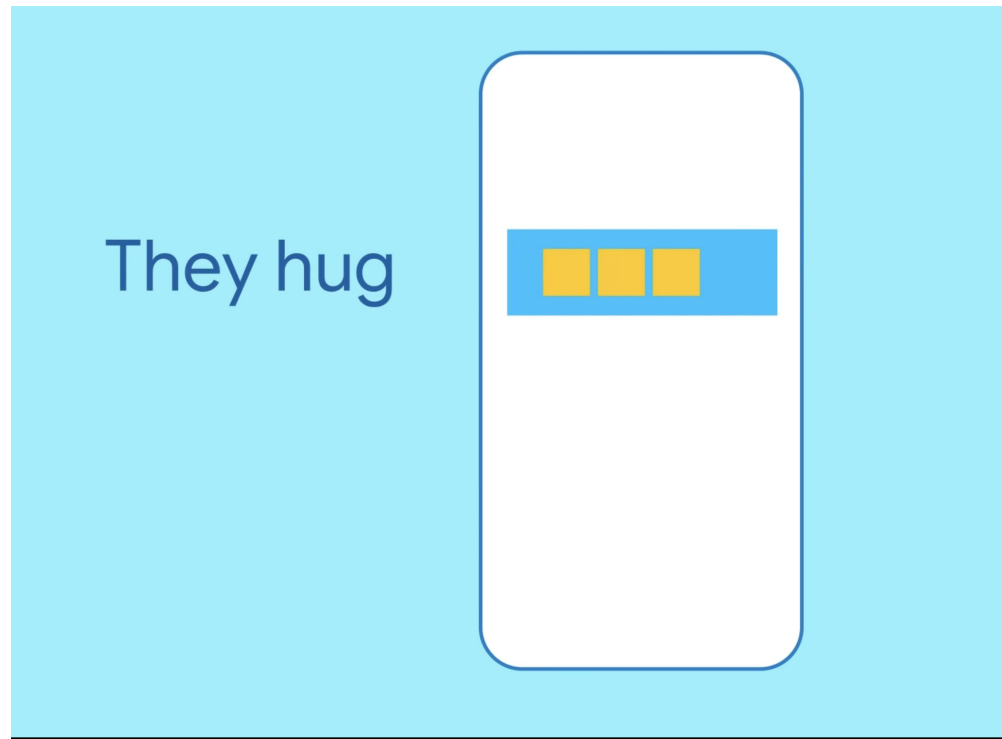
Creating Responsive Layouts using FlexBox and MediaQueries *(Expanded)*

Rows and columns
are pretty cool

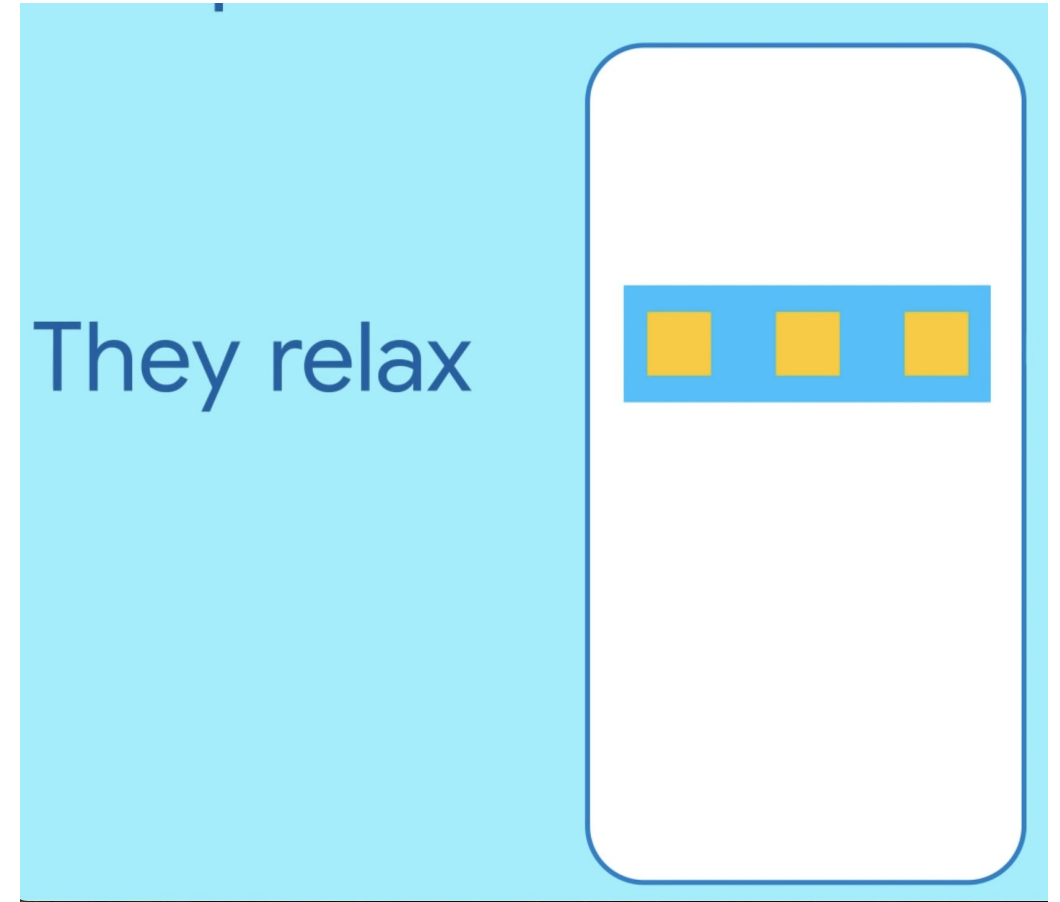


By default this is what rows and cols do.
But what if we would like to shape
each/specific element within them?

Creating Responsive Layouts using FlexBox and MediaQueries *(Expanded)*



They can hug



Or they can also relax

Creating Responsive Layouts using FlexBox and MediaQueries *(Expanded)*

Wrap it with
Expanded!

```
Row(  
  children: [  
    MyWidget(),  
    Expanded(  
      child: MyWidget()  
    ),  
    MyWidget(),  
  ],  
)
```

For Stretching, however, we need to wrap that with an Expanded Widget.

Creating Responsive Layouts using FlexBox and MediaQueries *(Expanded)*

The rest is for
flex widgets



If there are multiple elements you would like to stretch then, it determines the space with similar priority.

Creating Responsive Layouts using FlexBox and MediaQueries *(Expanded)*

Set a flex
factor

```
Expanded(  
  flex: 2,  
  child: Container()  
)
```

Setting Flex values to a specific element
Changes priorities on stretching them
accordingly.

Creating Responsive Layouts using FlexBox and MediaQueries *(Flexible)*

Flex values
determine the space

```
Column(children: [  
  Flexible(  
    flex: 2,  
    child: Container(color: Colors.cyan),  
  ),  
  Flexible(  
    flex: 3,  
    child: Container(color: Colors.teal),  
  ),  
  Flexible(  
    flex: 1,  
    child: Container(color: Colors.indigo),  
  ),  
]);
```

The flex values determine how much of each Flexible should take up space relative to its siblings.

Creating Responsive Layouts using FlexBox and MediaQueries *(Flexible)*

Flex values
determine the space



So here, the total number of flexes
Is $2 + 3 + 1 = 6$;
The first one as it has flex value of 2
Will take $2/6$ of space ($1/3$)
The next $3/6$ of space ($1/2$)
And the last $1/6$ of space

Creating Responsive Layouts using FlexBox and MediaQueries *(Flexible)*

Flexing strictness
is controlled by
the fit value

```
Column(children: [  
  Flexible(  
    flex: 2,  
    fit: FlexFit.tight,  
    child: Container(  
      height: 100,  
      color: Colors.cyan  
    ),  
  ),  
  Flexible(  
    flex: 3,  
    child: Container(color: Colors.teal),  
  ),  
  Flexible(  
    flex: 1,  
    child: Container(color: Colors.indigo),  
  ),  
]);
```

We could also use fit, to control how tight/loose we want our Flexible be.

Creating Responsive Layouts using FlexBox and MediaQueries *(Flexible)*

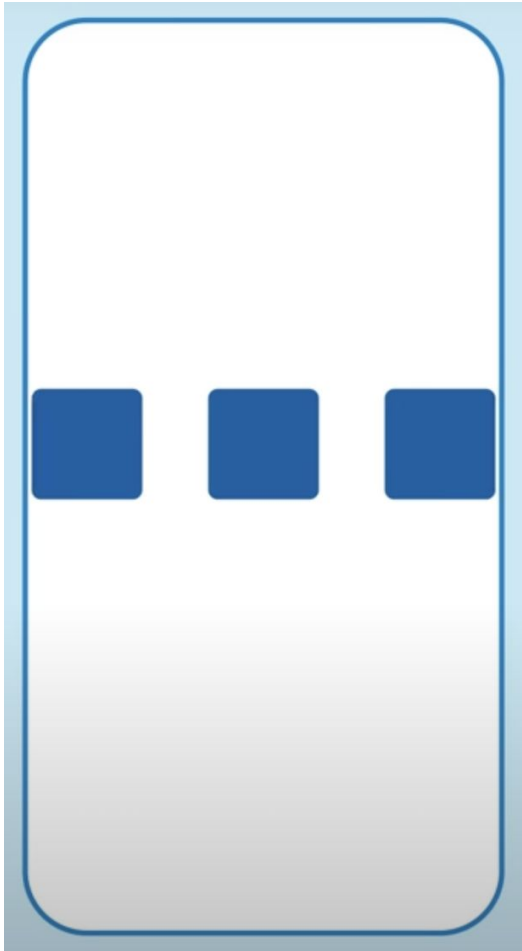
Flexing strictness
is controlled by
the fit value



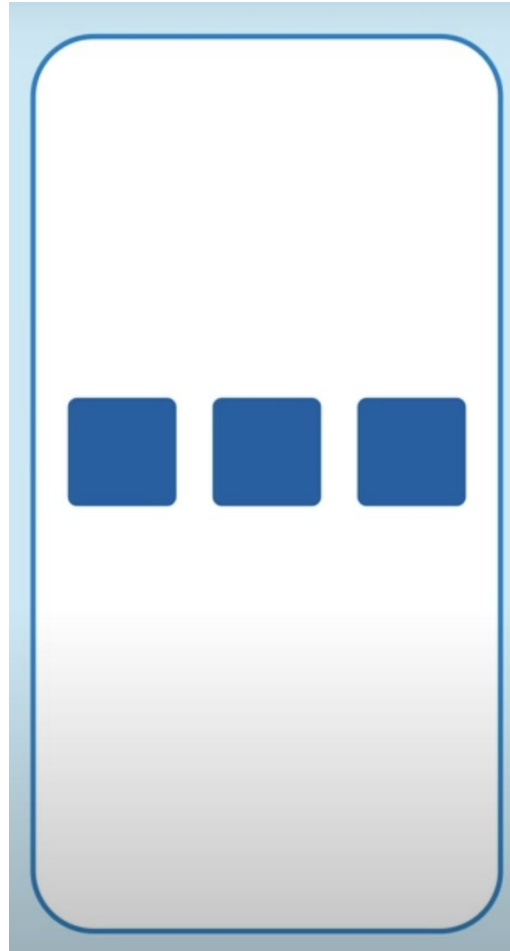
For instance, here the preferred tightness for the first Flexible is set to loose.

It is also possible to have fixed sized flexibles.

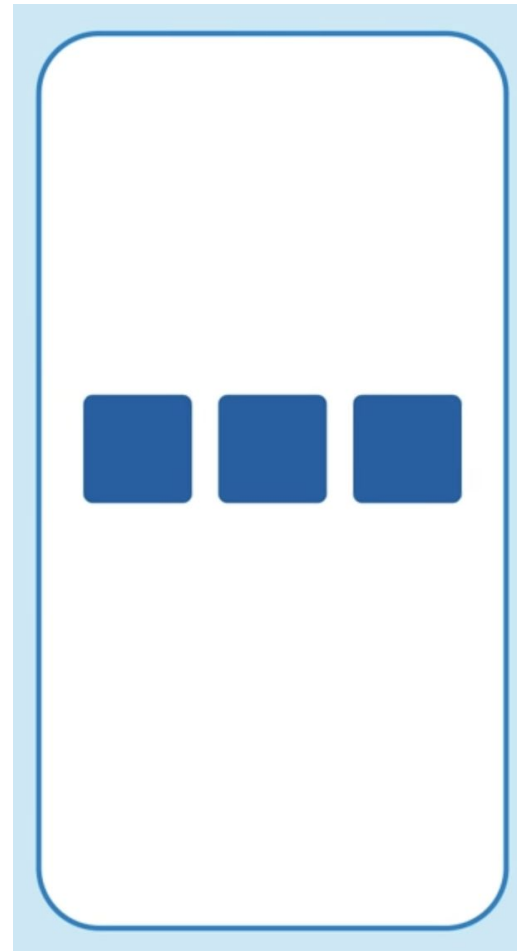
Creating Responsive Layouts using FlexBox and MediaQueries (P.S.: Spacer)



SpaceBetween



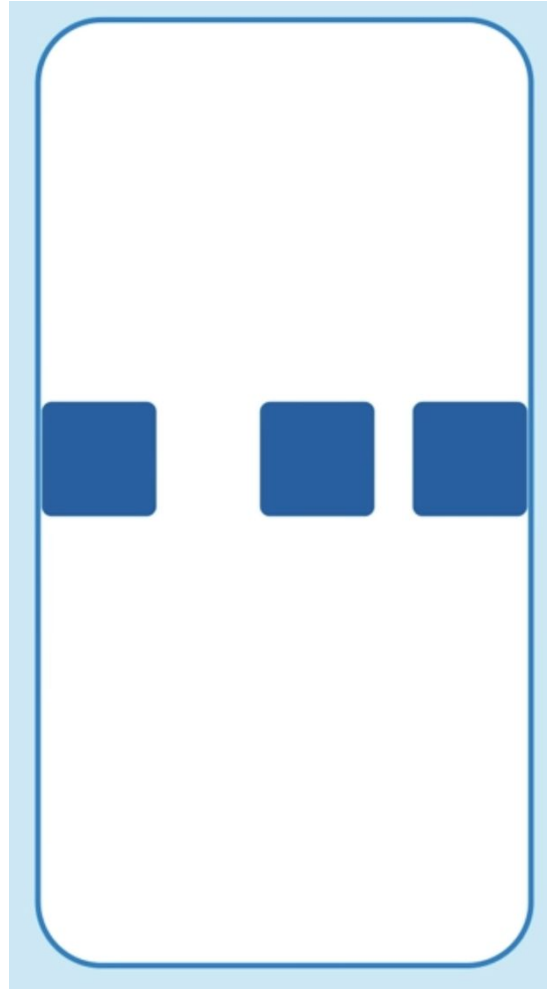
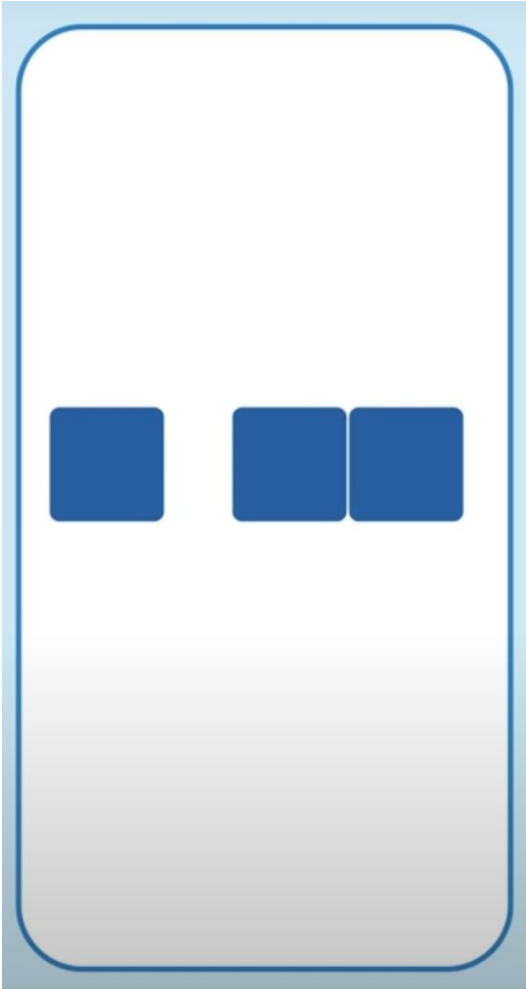
SpaceAround



SpaceEvenly

By default in Rows, and
Cols Widgets elements
layout as SpaceBetween
SpaceAround or
SpaceEvenly

Creating Responsive Layouts using FlexBox and MediaQueries (P.S.: Spacer)



If we would like to have Custom spaces in between Elements then, then we can Use Spacer in between them

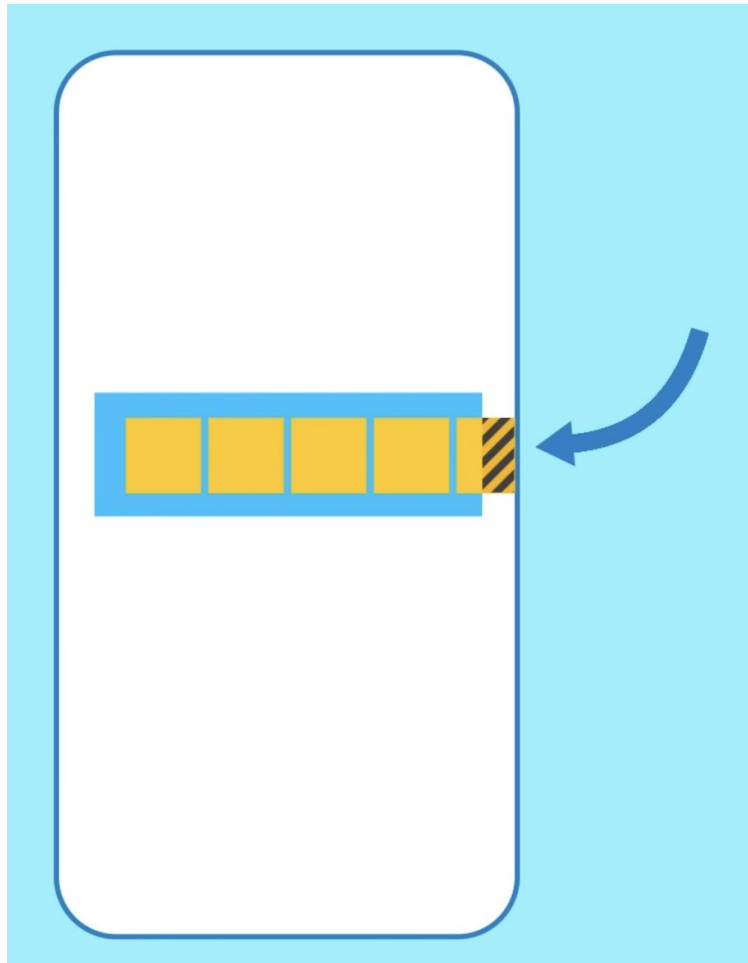
Creating Responsive Layouts using FlexBox and MediaQueries (*P.S.: Spacer*)

```
Row(  
  children: [  
    ColorBox(),  
    Spacer(),  
    ColorBox(),  
    Spacer(),  
    ColorBox(),  
  ],  
)
```

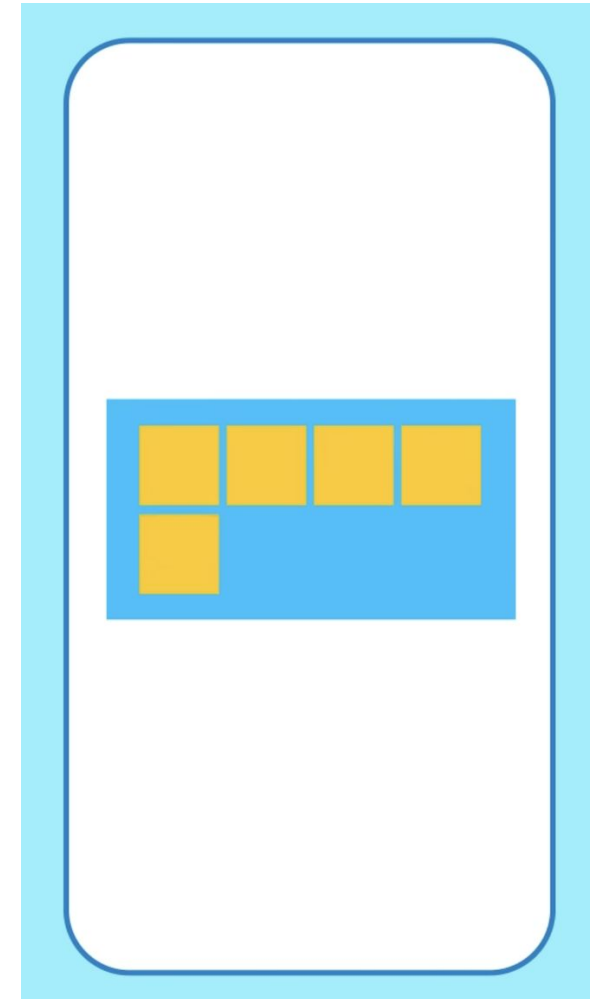
```
Row(  
  children: [  
    ColorBox(),  
    Spacer(flex: 2),  
    ColorBox(),  
    Spacer(),  
    ColorBox(),  
  ],  
)
```

```
Row(  
  children: [  
    ColorBox(),  
    Spacer(),  
    ColorBox(),  
    Spacer(flex:3),  
    ColorBox(),  
    Spacer(flex:9),  
    ColorBox(),  
  ],  
)
```

Creating Responsive Layouts using FlexBox and MediaQueries (*P.S.: Wrap*)



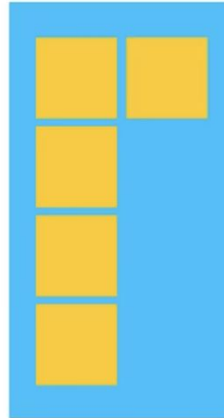
```
Wrap(  
  children: [  
    MyWidget(),  
    MyWidget(),  
    MyWidget(),  
  ],  
)
```



Wrapping makes
Elements fit in
The screen.

Creating Responsive Layouts using FlexBox and MediaQueries (*P.S.: Wrap*)

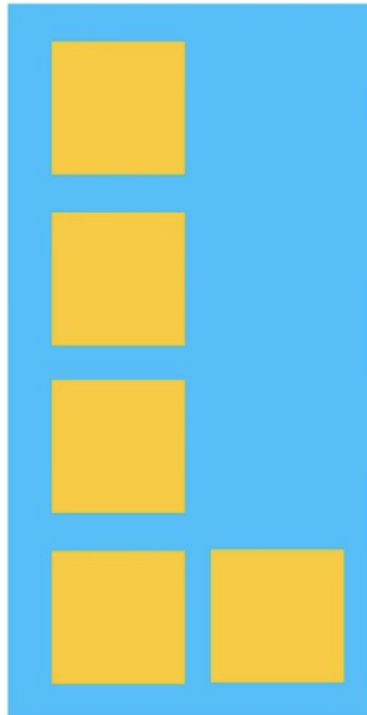
- You can also control Axis as Vertical or horizontal using direction



```
Wrap(  
  direction: Axis.vertical,  
  children: [  
    MyWidget(),  
    MyWidget(),  
    MyWidget(),  
    MyWidget(),  
    MyWidget(),  
  ],  
)
```

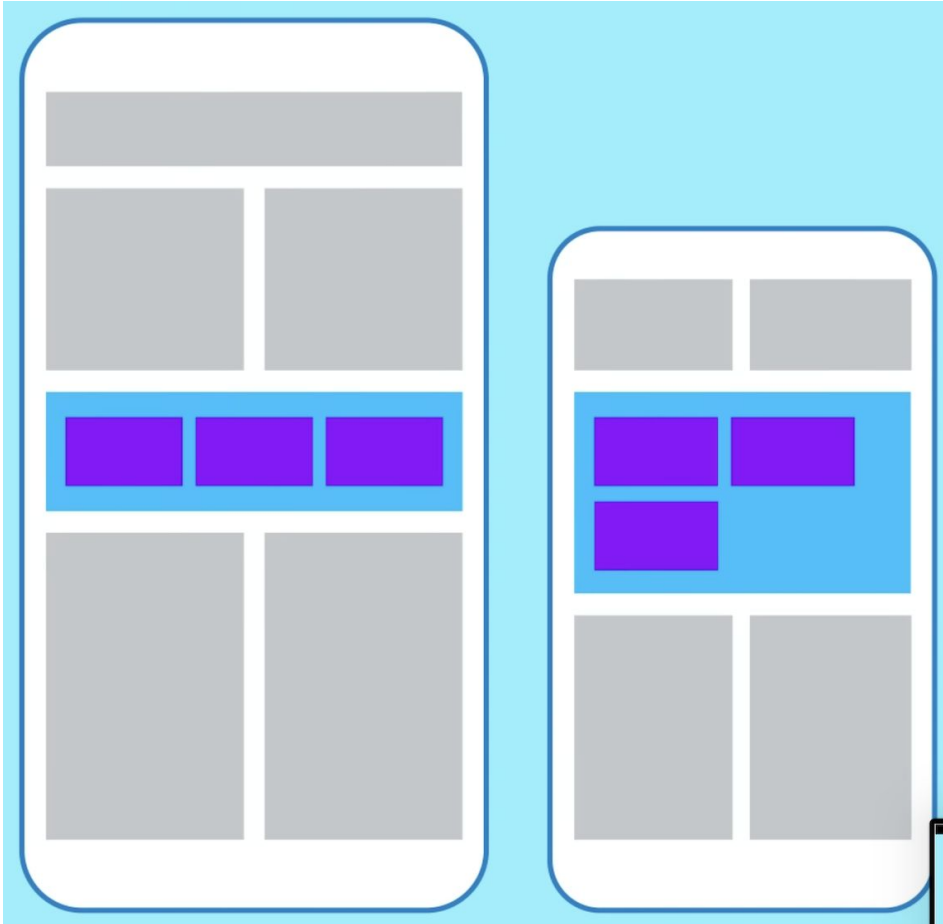
Creating Responsive Layouts using FlexBox and MediaQueries (*P.S.: Wrap*)

- You can also control Axis as Vertical or horizontal using direction
- Also, control alignment
- As well as spacing

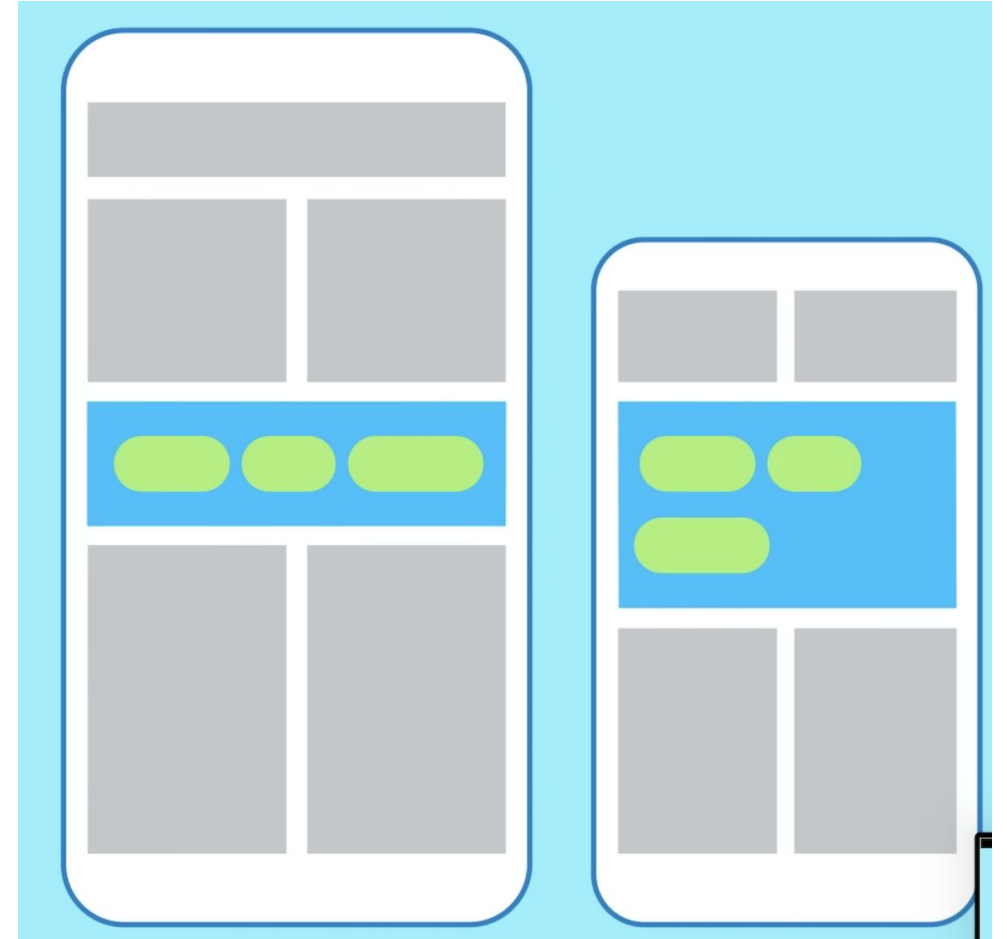


```
Wrap(  
  alignment: WrapAlignment.end,  
  spacing: 10.0,  
  runSpacing: 20.0,  
  children: [  
    MyWidget(),  
    MyWidget(),  
    MyWidget(),  
    MyWidget(),  
    MyWidget(),  
  ],  
)
```

Creating Responsive Layouts using FlexBox and MediaQueries (*P.S.: Wrap*)



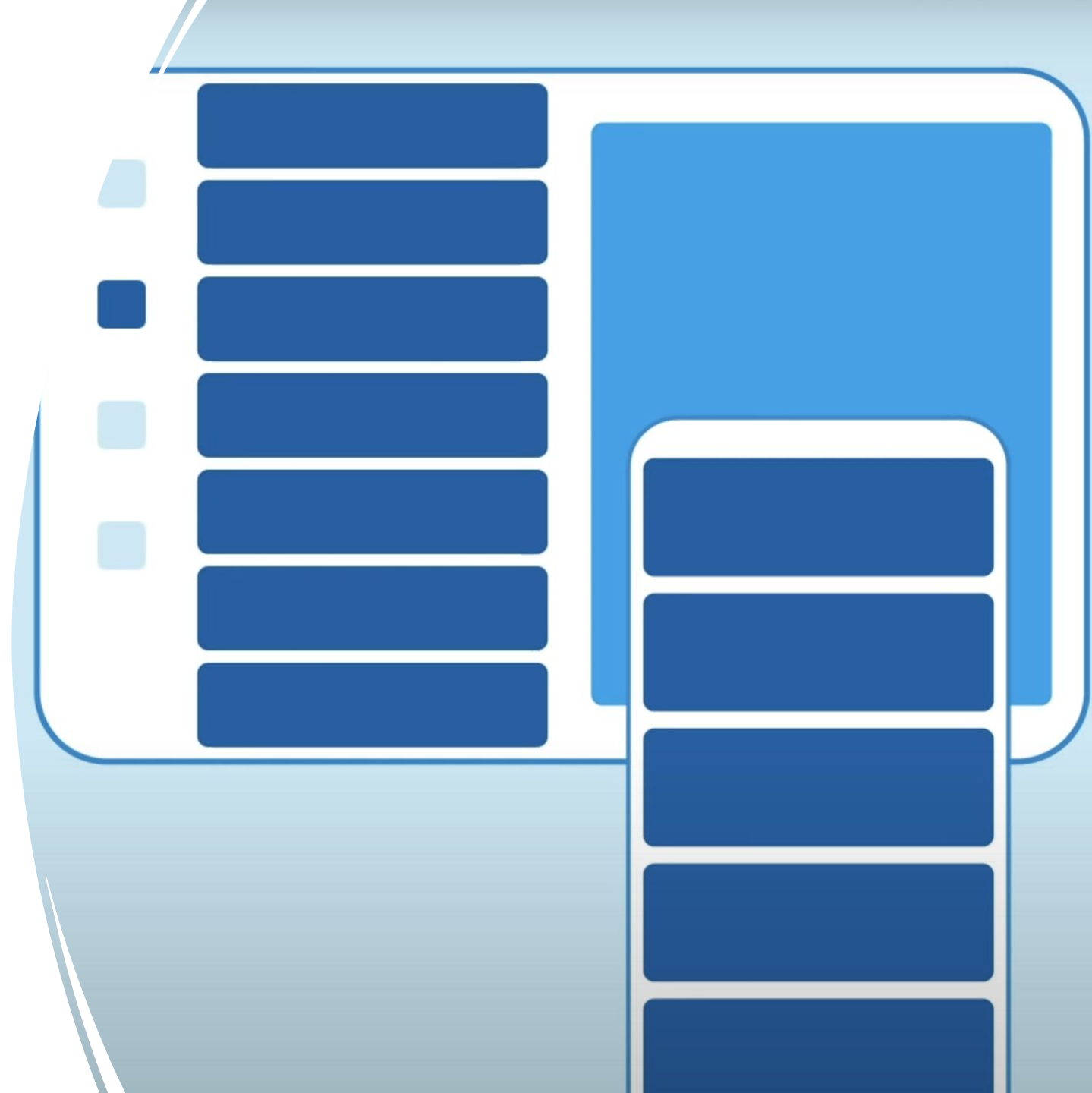
With buttons



With chips

Media Queries in Flutter (*use cases*)

If you are making an app for different screen layout devices such as Cell phone or a tablet. Then you would want to have different set of information on these screens.



Media Query in Flutter

- Media Query is a more complex form of Layout Builder.
- It contains many handy features like screen orientation, screen size, or the size of any widget.

```
build(BuildContext context) {  
    var screenSize =  
        MediaQuery.of(context).size;  
    if (screenSize.width > oneColumnLayout) {  
  
    } else {  
        // one column layout  
    }  
}
```

LayoutBuilder

<https://api.flutter.dev/flutter/widgets/LayoutBuilder-class.html>

Impeller Rendering Engine

<https://docs.flutter.dev/perf/impeller>

Web Assembly (wasm)

<https://docs.flutter.dev/platform-integration/web/wasm>

Want a Challenge??

(make the Attendance Sheet Widget)

Using flutter.dev and an online compiler try to come up with a unique widget with the creation of your first Widget by combining already existing ones from the Documentation (flutter.dev)!